

Introduction

In today's cybersecurity landscape, networks generate an overwhelming amount of traffic, making it unrealistic for security analysts to manually review every connection. Traditional intrusion detection systems (IDS) often rely on known signatures, which means they are effective at identifying previously seen attacks but may struggle to detect new or evolving threats. This creates a gap where malicious activity can go unnoticed simply because it does not match an existing pattern. The goal of this project is to explore how machine learning can be used to improve intrusion detection by identifying patterns in network traffic behavior rather than relying solely on predefined signatures.

Dataset

For this project, I used the CICIDS2017 dataset, which was created to simulate realistic network traffic. It includes both normal activity and several types of modern cyberattacks. Out of the full dataset, I focused specifically on the Thursday-WorkingHours-Morning-WebAttacks subset. This file contains 458,968 records and 85 attributes. Each row represents a network flow, which can be thought of as a summarized version of a communication session between two systems. Instead of analyzing every packet individually, the dataset captures things like how long the communication lasted, how much data was transferred, and how the traffic behaved over time. The subset I used includes benign traffic along with three types of web attacks: brute force, cross-site scripting (XSS), and SQL injection. These are all common attack types in real environments, especially when dealing with web applications, so it made sense to focus on this portion of the data.

Methodology

Before training any models, I fully cleaned and prepared the dataset. This included fixing encoding issues when loading the file, removing duplicate columns, handling missing values, and cleaning up the labels so they were consistent. I also converted the problem into a binary classification task, where all attack types were grouped into one category. That made it easier to focus on the core question: can the model tell the difference between normal traffic and malicious traffic? One issue I ran into early was class imbalance. There were significantly more benign records than attack records, which can cause models to just predict everything as benign. To fix that, I used undersampling to balance the dataset, ending up with 4,360 records split evenly between the two classes. Another important part of the process was making sure the model wasn't taking shortcuts. During early testing, I noticed that one model was performing extremely well. After taking a closer look and also discussing it with members of a security team, it became clear that it was relying heavily on connection related attributes instead of actual traffic behavior. Based on that, I adjusted the approach to focus more on behavioral features like packet size, flow rate, and timing. The goal was to make sure the model was learning something meaningful, not just memorizing identifiers. To get a better comparison, I trained multiple models, including Decision Tree, Random Forest, Logistic Regression, Linear SVM, K-Nearest Neighbors (KNN), Naive Bayes, and Linear Discriminant Analysis (LDA). Each of these models approaches the problem differently, which helped give a more complete picture of performance.

Results

Overall, the results were strong across most models. Random Forest performed the best, with an accuracy of about 97.9%, along with strong precision, recall, and F1-score. Logistic Regression and KNN also performed very well, which was interesting because they are simpler models compared to Random Forest. This suggests that once the data is properly cleaned and prepared, the patterns in the dataset are fairly clear and can be learned by different types of models. Naive Bayes stood out the most. Its accuracy was lower, around 86.9%, but its recall was extremely high, close to 99.7%. That means it was able to catch almost every attack, but it also came with a tradeoff. Its precision was only around 79%, which means it was flagging a lot of normal traffic as malicious. The other models, including Linear SVM, Decision Tree, and LDA, all performed in a solid range as well. Seeing multiple models perform well gave more confidence that the results weren't tied to just one specific approach.

Real-World Security Insight

Once I looked at the results from a real-world perspective, it became clear that the numbers alone don't tell the full story. In a SOC environment, it's not just about how many attacks a model can catch. It's also about how usable the model is for analysts. For example, Naive Bayes had extremely high recall, which sounds great at first. But because of its lower precision, it would generate a large number of false positives. Based on conversations with a security team, this is a serious issue in practice. If analysts are constantly dealing with alerts that turn out to be nothing, they can start to lose confidence in the system and may begin ignoring alerts altogether. At the same time, that doesn't mean a model like Naive Bayes has no value. It could still be useful as an early detection layer where the goal is to catch as much as possible before passing results to a more precise model. This made it clear that the "best" model depends on how it's actually going to be used. A model like Random Forest stood out because it provided a strong balance between catching attacks and avoiding unnecessary noise, which is more aligned with how security teams operate.

Ethical Considerations

Using machine learning in cybersecurity also comes with responsibility. One important factor is how the model handles mistakes. Missing an attack can lead to real damage, but too many false positives can overwhelm analysts and reduce trust in the system. There's also the issue of data privacy. Network traffic data can contain sensitive information, so it needs to be handled carefully throughout the process. Another thing to consider is that models can learn patterns that don't always generalize well outside the dataset. That means even if a model performs well here, it still needs to be monitored and evaluated if it were ever used in a real environment. Because of that, machine learning should be seen as a tool to support analysts, not replace them. Human judgment is still necessary to interpret and act on the results.

Conclusion

This project demonstrates that machine learning can be effectively applied to network intrusion detection by focusing on behavioral patterns in traffic data. By preprocessing the CICIDS2017 dataset, balancing the classes, and evaluating multiple models, I was able to identify approaches that perform well both statistically and practically. Random Forest proved to be the most effective overall model due to its balanced performance, while other models provided valuable insight into different tradeoffs between precision and recall. Overall, this project highlights how machine learning can enhance

intrusion detection by helping analysts identify suspicious activity more efficiently, while also emphasizing the importance of thoughtful model selection and responsible implementation.

```
# Core libraries for data handling & preprocessing
import os
import random
import zipfile
import numpy as np
import pandas as pd

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Encoding / preprocessing
from sklearn import preprocessing

%matplotlib inline
```

```
# Set dataset path
DATA_FILE = "/Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv"

# Confirm file exists
print("DATA_FILE exists:", os.path.exists(DATA_FILE))

if os.path.exists(DATA_FILE):
    print("\nLoaded file:")
    print(os.path.basename(DATA_FILE))
```

DATA_FILE exists: True

Loaded file:
Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv

```
# Load dataset into DataFrame

df_raw = pd.read_csv(DATA_FILE, encoding="latin-1", engine="python")

# Basic shape check
print("Dataset shape:", df_raw.shape)

# Preview first few rows
df_raw.head()
```

Dataset shape: (458968, 85)

	Flow ID	Source IP	Source Port	Destination IP	Destination Port	Protocol	Timestamp	Dr
0	192.168.10.3-192.168.10.50-389-33898-6	192.168.10.50	33898.0	192.168.10.3	389.0	6.0	6/7/2017 8:59	1130
1	192.168.10.3-192.168.10.50-389-33904-6	192.168.10.50	33904.0	192.168.10.3	389.0	6.0	6/7/2017 8:59	1134
2	8.0.6.4-8.6.0.1-0-0-0	8.6.0.1	0.0	8.0.6.4	0.0	0.0	6/7/2017 8:59	1199
3	192.168.10.14-65.55.44.109-59135-443-6	192.168.10.14	59135.0	65.55.44.109	443.0	6.0	6/7/2017 8:59	602
4	192.168.10.3-192.168.10.14-53-59555-17	192.168.10.14	59555.0	192.168.10.3	53.0	17.0	6/7/2017 8:59	

5 rows x 85 columns

```
# Clean column names (remove spaces)
df = df_raw.copy()
df.columns = df.columns.str.strip()

# Remove duplicate column
if "Fwd Header Length.1" in df.columns:
    df = df.drop(columns=["Fwd Header Length.1"])

print("Shape after cleaning:", df.shape)
print("Flow ID" in df.columns)
print(df.columns[:10].tolist())
```

Shape after cleaning: (458968, 84)
True
['Flow ID', 'Source IP', 'Source Port', 'Destination IP', 'Destination Port', 'Protoc

```
# Check labels
print("Unique labels:")
print(df["Label"].unique())

print("\nLabel distribution:")
print(df["Label"].value_counts())
```

Unique labels:
['BENIGN' 'Web Attack \x96 Brute Force' 'Web Attack \x96 XSS'
 'Web Attack \x96 Sql Injection' nan]

Label distribution:
Label
BENIGN 168186
Web Attack Brute Force 1507
Web Attack XSS 652

```
Web Attack    Sql Injection    21
Name: count, dtype: int64
```

```
# Fix label text encoding issues
df["Label"] = df["Label"].str.replace("\x96", "-", regex=False)

# Strip extra spaces
df["Label"] = df["Label"].str.strip()

print("Cleaned labels:")
print(df["Label"].unique())
```

```
Cleaned labels:
['BENIGN' 'Web Attack - Brute Force' 'Web Attack - XSS'
 'Web Attack - Sql Injection' nan]
```

```
# Remove rows with missing Flow ID
df = df.drop(df[pd.isnull(df["Flow ID"])]).index)

print("Shape after removing empty rows:", df.shape)
```

```
Shape after removing empty rows: (170366, 84)
```

```
# Replace "Infinity"
df.replace("Infinity", -1, inplace=True)

# Convert to numeric
df[["Flow Bytes/s", "Flow Packets/s"]] = df[["Flow Bytes/s", "Flow Packets/s"]].appl

# Replace NaN / inf
df.replace([np.inf, -np.inf, np.nan], -1, inplace=True)

print("Remaining missing values:", df.isna().sum().sum())
```

```
Remaining missing values: 0
```

```
from sklearn import preprocessing

# Identify categorical columns
string_features = list(df.select_dtypes(include=["object"]).columns)
string_features.remove("Label")

print("Encoding columns:", string_features)

# Encode
le = preprocessing.LabelEncoder()

for col in string_features:
    df[col] = le.fit_transform(df[col])

Encoding columns: ['Flow ID', 'Source IP', 'Destination IP', 'Timestamp']
```

```
# Convert labels: BENIGN = 0, ATTACK = 1
df["Label"] = df["Label"].apply(lambda x: 0 if x == "BENIGN" else 1)
```

```
print("Updated distribution:")
print(df["Label"].value_counts())
```

```
Updated distribution:
Label
0      168186
1       2180
Name: count, dtype: int64
```

```
# Analyze class imbalance
```

```
benign_total = len(df[df["Label"] == 0])
attack_total = len(df[df["Label"] == 1])
```

```
print("Total BENIGN:", benign_total)
print("Total ATTACK:", attack_total)
```

```
Total BENIGN: 168186
Total ATTACK: 2180
```

```
# Undersampling to balance dataset
```

```
# Target: similar number of benign and attack samples
```

```
df_benign = df[df["Label"] == 0]
df_attack = df[df["Label"] == 1]
```

```
# Sample benign to match attack count
```

```
df_benign_sample = df_benign.sample(n=attack_total, random_state=42)
```

```
# Combine datasets
```

```
df_balanced = pd.concat([df_benign_sample, df_attack])
```

```
# Shuffle dataset
```

```
df_balanced = df_balanced.sample(frac=1, random_state=42).reset_index(drop=True)
```

```
print("Balanced dataset shape:", df_balanced.shape)
print("\nBalanced class distribution:")
print(df_balanced["Label"].value_counts())
```

```
Balanced dataset shape: (4360, 84)
```

```
Balanced class distribution:
Label
0       2180
1       2180
Name: count, dtype: int64
```

```
# Drop features that can cause unrealistic learning
```

```
excluded = [
    "Flow ID",
    "Source IP",
    "Source Port",
    "Destination IP",
    "Destination Port",
    "Protocol",
    "Timestamp"
]
```

```
df_balanced = df_balanced.drop(columns=excluded, errors="ignore")

# Drop suspicious features
df_balanced = df_balanced.drop(
    columns=["Init_Win_bytes_backward", "Init_Win_bytes_forward"],
    errors="ignore"
)

print("Shape after dropping leakage features:", df_balanced.shape)
```

```
Shape after dropping leakage features: (4360, 75)
```

When I first ran the Decision Tree and cross-validation, the model produced near-perfect scores, which immediately stood out as unrealistic. After looking closer at the tree, I noticed it was relying heavily on features like Source IP and Source Port, which essentially allowed the model to memorize patterns rather than actually learn what makes traffic malicious. To fix this, I removed those identifier-based features along with the Init_Win_bytes variables, since they can introduce data leakage or overly strong signals. After making these changes and rerunning the model, the cross-validation scores dropped to a more realistic range (around 0.92–0.94), and the tree began focusing on behavioral features like packet length and flow characteristics. This confirmed that the model is now learning meaningful patterns instead of overfitting to specific identifiers.

```
from sklearn.model_selection import train_test_split

X = df_balanced.drop(columns=["Label"])
y = df_balanced["Label"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=42
)

print("Train:", X_train.shape)
print("Test:", X_test.shape)
```

```
Train: (3052, 74)
Test: (1308, 74)
```

Decision Tree

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score

# Simple tree for interpretation
decision_tree = DecisionTreeClassifier(max_leaf_nodes=5, random_state=0)

decision_tree.fit(X_train, y_train)

# Cross-validation score
scores = cross_val_score(decision_tree, X_train, y_train, cv=10)
```

```
print("Cross-validation scores:", scores)
print("Average score:", scores.mean())
```

```
Cross-validation scores: [0.94117647 0.91830065 0.93770492 0.92786885 0.92131148 0.94
0.91803279 0.91803279 0.9147541 0.93770492]
Average score: 0.9275870566805958
```

```
from sklearn.tree import export_text
```

```
tree_rules = export_text(decision_tree, feature_names=list(X_train.columns))
print(tree_rules)
```

```
|--- Max Packet Length <= 3.00
|   |--- Fwd Header Length <= 100.00
|   |   |--- class: 1
|   |--- Fwd Header Length > 100.00
|   |   |--- class: 1
|--- Max Packet Length > 3.00
|   |--- Subflow Fwd Bytes <= 30857.00
|   |   |--- Bwd IAT Min <= 841.00
|   |   |   |--- class: 0
|   |   |--- Bwd IAT Min > 841.00
|   |   |   |--- class: 0
|   |--- Subflow Fwd Bytes > 30857.00
|   |   |--- class: 1
```

```
import numpy as np
```

```
importances = decision_tree.feature_importances_
indices = np.argsort(importances)[::-1]
```

```
print("Top Decision Tree Features:\n")
```

```
for i in indices[:10]:
    print(f"{X_train.columns[i]}: {importances[i]:.4f}")
```

```
Top Decision Tree Features:
```

```
Max Packet Length: 0.8077
Subflow Fwd Bytes: 0.0792
Fwd Header Length: 0.0777
Bwd IAT Min: 0.0353
Active Min: 0.0000
Active Max: 0.0000
Idle Std: 0.0000
Idle Mean: 0.0000
Idle Min: 0.0000
Idle Max: 0.0000
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
```

```
# Decision Tree predictions
y_pred_dt = decision_tree.predict(X_test)
```

```
# Decision Tree metrics
accuracy_dt = accuracy_score(y_test, y_pred_dt)
precision_dt = precision_score(y_test, y_pred_dt)
recall_dt = recall_score(y_test, y_pred_dt)
```

```
f1_dt = f1_score(y_test, y_pred_dt)

print("Decision Tree Results:")
print("Accuracy =", accuracy_dt)
print("Precision =", precision_dt)
print("Recall =", recall_dt)
print("F1 =", f1_dt)

print("\nDecision Tree Confusion Matrix:")
print(confusion_matrix(y_test, y_pred_dt))
```

```
Decision Tree Results:
Accuracy = 0.9250764525993884
Precision = 0.9007299270072993
Recall = 0.9536321483771252
F1 = 0.9264264264264265

Decision Tree Confusion Matrix:
[[593  68]
 [ 30 617]]
```

Random Forest

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=250,
    random_state=42,
    oob_score=True
)

rf.fit(X_train, y_train)

print("Training accuracy:", rf.score(X_train, y_train))
print("Test accuracy:", rf.score(X_test, y_test))
print("OOB score:", rf.oob_score_)
```

```
Training accuracy: 0.9921363040629095
Test accuracy: 0.9785932721712538
OOB score: 0.9790301441677588
```

```
importances = rf.feature_importances_
indices = np.argsort(importances)[::-1]

print("\nTop 10 Important Features:\n")

for i in indices[:10]:
    print(f"{X.columns[i]}: {importances[i]:.4f}")
```

```
Top 10 Important Features:

Average Packet Size: 0.0943
Flow Bytes/s: 0.0784
Max Packet Length: 0.0659
Fwd Packet Length Mean: 0.0644
Packet Length Mean: 0.0609
Fwd IAT Min: 0.0580
```

```
Subflow Fwd Bytes: 0.0575
Avg Fwd Segment Size: 0.0537
Total Length of Fwd Packets: 0.0422
Fwd Packet Length Max: 0.0350
```

Random Forest

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,

y_pred = rf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("Accuracy =", accuracy)
print("Precision =", precision)
print("Recall =", recall)
print("F1 =", f1)

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
Accuracy = 0.9785932721712538
Precision = 0.9768875192604006
Recall = 0.9799072642967542
F1 = 0.9783950617283951
```

```
Confusion Matrix:
[[646  15]
 [ 13 634]]
```

At this stage, I trained a Random Forest model to evaluate how well the dataset could be used to distinguish between benign and malicious network traffic. The model achieved strong performance across all evaluation metrics, with accuracy, precision, recall, and F1-score all around 97–98%. These results indicate that the model is effectively learning patterns in the data, particularly behavioral characteristics such as packet size, flow rate, and timing features. This is important because it shows the model is not relying on easily spoofed identifiers like IP addresses, but instead focusing on how the traffic behaves.

Logistic Regression

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Train Logistic Regression
lr = LogisticRegression(max_iter=100000)

lr.fit(X_train, y_train)

# Predictions
y_pred_lr = lr.predict(X_test)
```

```
# Metrics
accuracy_lr = accuracy_score(y_test, y_pred_lr)
precision_lr = precision_score(y_test, y_pred_lr)
recall_lr = recall_score(y_test, y_pred_lr)
f1_lr = f1_score(y_test, y_pred_lr)

print("Logistic Regression Results:")
print("Accuracy =", accuracy_lr)
print("Precision =", precision_lr)
print("Recall =", recall_lr)
print("F1 =", f1_lr)
```

```
Logistic Regression Results:
Accuracy = 0.97782874617737
Precision = 0.9681818181818181
Recall = 0.9876352395672334
F1 = 0.9778117827084928
/usr/local/lib/python3.13/dist-packages/sklearn/linear_model/_logistic.py:465: Conver
STOP: TOTAL NO. OF F,G EVALUATIONS EXCEEDS LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:
<https://scikit-learn.org/stable/modules/preprocessing.html>
 Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
 n_iter_i = _check_optimize_result(

At this stage, I tested multiple models to get a better sense of how well this dataset can be used for intrusion detection. I started with a Decision Tree because it's easy to interpret, and it helped show how the model makes decisions. From that, I was able to see that features like packet length and timing played a big role in identifying malicious traffic. However, its performance was a bit lower and it relied heavily on simple splits. When I moved to the Random Forest model, the performance improved significantly, with accuracy, precision, recall, and F1-score all around 97–98%. This showed that using multiple trees together helps capture more complex patterns in the data. I also ran Logistic Regression as a comparison, and it performed very close to the Random Forest, which was interesting. That suggests the dataset has strong patterns that can even be picked up by a simpler linear model. I did run into a convergence warning with Logistic Regression, even after increasing the number of iterations, but the results were still stable and consistent, so the model was clearly still learning useful patterns. Overall, these results show that the dataset works well for machine learning-based intrusion detection, especially when focusing on behavioral features like packet size, flow rate, and timing. Moving forward, I plan to test a few additional models to compare results and better understand the strengths and limitations of each approach.

SVM

```
from sklearn.svm import LinearSVC

svm = LinearSVC(max_iter=5000)

svm.fit(X_train, y_train)

y_pred_svm = svm.predict(X_test)
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

accuracy_svm = accuracy_score(y_test, y_pred_svm)
precision_svm = precision_score(y_test, y_pred_svm)
recall_svm = recall_score(y_test, y_pred_svm)
f1_svm = f1_score(y_test, y_pred_svm)

print("SVM (LinearSVC) Results:")
print("Accuracy =", accuracy_svm)
print("Precision =", precision_svm)
print("Recall =", recall_svm)
print("F1 =", f1_svm)
```

```
SVM (LinearSVC) Results:
Accuracy = 0.9709480122324159
Precision = 0.9551569506726457
Recall = 0.9876352395672334
F1 = 0.9711246200607903
```

I also wanted to include a Support Vector Machine (SVM) model since it's commonly used for classification problems like this. However, when I tried running the standard SVM, it ended up taking over an hour without finishing, which immediately stood out to me. That made it clear that while SVMs can be powerful, they're not always practical for larger datasets like this one. In a real-world setting, especially in cybersecurity, speed matters just as much as accuracy. Because of that, I switched to a more efficient version, LinearSVC. This allowed me to still test the SVM approach but in a way that actually runs in a reasonable time. The results were still strong, which confirmed that this was a better fit for the size and structure of the dataset.

KNN

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=5)

knn.fit(X_train, y_train)

y_pred_knn = knn.predict(X_test)

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

accuracy_knn = accuracy_score(y_test, y_pred_knn)
precision_knn = precision_score(y_test, y_pred_knn)
recall_knn = recall_score(y_test, y_pred_knn)
f1_knn = f1_score(y_test, y_pred_knn)

print("KNN Results:")
print("Accuracy =", accuracy_knn)
print("Precision =", precision_knn)
print("Recall =", recall_knn)
print("F1 =", f1_knn)
```

```
KNN Results:
Accuracy = 0.9724770642201835
Precision = 0.9607843137254902
```

```
Recall = 0.9845440494590417  
F1 = 0.9725190839694656
```

I also implemented a K-Nearest Neighbors (KNN) model to include a distance-based approach in the analysis. Unlike the other models, KNN makes predictions based on how similar a data point is to its neighbors rather than learning explicit decision rules. The model performed well overall, with an accuracy of about 97.25% and a strong balance between precision and recall. In particular, the recall was very high at around 98.45%, which shows that the model is effective at identifying attack traffic. These results suggest that malicious and benign traffic patterns are distinct enough for similarity-based methods to work effectively on this dataset.

Naive Bayes

```
from sklearn.naive_bayes import GaussianNB  
  
nb = GaussianNB()  
  
nb.fit(X_train, y_train)  
  
y_pred_nb = nb.predict(X_test)  
  
accuracy_nb = accuracy_score(y_test, y_pred_nb)  
precision_nb = precision_score(y_test, y_pred_nb)  
recall_nb = recall_score(y_test, y_pred_nb)  
f1_nb = f1_score(y_test, y_pred_nb)  
  
print("Naive Bayes Results:")  
print("Accuracy =", accuracy_nb)  
print("Precision =", precision_nb)  
print("Recall =", recall_nb)  
print("F1 =", f1_nb)
```

```
Naive Bayes Results:  
Accuracy = 0.8692660550458715  
Precision = 0.7923832923832924  
Recall = 0.9969088098918083  
F1 = 0.8829568788501027
```

I included a Naive Bayes model as a simple probabilistic approach to classification. This model assumes that features are independent, which is not always realistic for network traffic data. As expected, the performance was lower compared to the other models, with an accuracy of about 86.93%. However, one interesting result was the very high recall of approximately 99.69%, meaning the model was able to detect almost all attack instances. The trade-off was lower precision, which indicates more false positives. This shows that while Naive Bayes may not be as precise, it can still be useful in scenarios where catching as many attacks as possible is the priority.

LDA

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
```

```
lda = LinearDiscriminantAnalysis()

lda.fit(X_train, y_train)

y_pred_lda = lda.predict(X_test)

accuracy_lda = accuracy_score(y_test, y_pred_lda)
precision_lda = precision_score(y_test, y_pred_lda)
recall_lda = recall_score(y_test, y_pred_lda)
f1_lda = f1_score(y_test, y_pred_lda)

print("LDA Results:")
print("Accuracy =", accuracy_lda)
print("Precision =", precision_lda)
print("Recall =", recall_lda)
print("F1 =", f1_lda)
```

```
LDA Results:
Accuracy = 0.9403669724770642
Precision = 0.956661316211878
Recall = 0.9211746522411128
F1 = 0.9385826771653544
```

LDA works by finding a linear combination of features that best separates the classes. The model performed well, achieving an accuracy of about 94.04%, along with strong precision and recall. While it did not outperform the top models like Random Forest or Logistic Regression, it still demonstrated that the dataset contains patterns that can be separated using relatively simple linear techniques.

```
import pandas as pd

# Build final model comparison table
results_df = pd.DataFrame([
    {
        "Model": "Decision Tree",
        "Accuracy": 0.9250764525993884,
        "Precision": 0.9007299270072993,
        "Recall": 0.9536321483771252,
        "F1": 0.9264264264264265
    },
    {
        "Model": "Random Forest",
        "Accuracy": 0.9785932721712538,
        "Precision": 0.9768875192604006,
        "Recall": 0.9799072642967542,
        "F1": 0.9783950617283951
    },
    {
        "Model": "Logistic Regression",
        "Accuracy": 0.97782874617737,
        "Precision": 0.9681818181818181,
        "Recall": 0.9876352395672334,
        "F1": 0.9778117827084928
    },
    {
        "Model": "Linear SVM",
        "Accuracy": 0.9709480122324159,
```

```
"Precision": 0.9551569506726457,
"Recall": 0.9876352395672334,
"F1": 0.9711246200607903
},
{
  "Model": "KNN",
  "Accuracy": 0.9724770642201835,
  "Precision": 0.9607843137254902,
  "Recall": 0.9845440494590417,
  "F1": 0.9725190839694656
},
{
  "Model": "Naive Bayes",
  "Accuracy": 0.8692660550458715,
  "Precision": 0.7923832923832924,
  "Recall": 0.9969088098918083,
  "F1": 0.8829568788501027
},
{
  "Model": "LDA",
  "Accuracy": 0.9403669724770642,
  "Precision": 0.956661316211878,
  "Recall": 0.9211746522411128,
  "F1": 0.9385826771653544
}
]

# Round for cleaner display
results_df_rounded = results_df.copy()
metric_cols = ["Accuracy", "Precision", "Recall", "F1"]
results_df_rounded[metric_cols] = results_df_rounded[metric_cols].round(4)

print("Final Model Comparison Table:")
display(results_df_rounded)
```

Final Model Comparison Table:

	Model	Accuracy	Precision	Recall	F1	
0	Decision Tree	0.9251	0.9007	0.9536	0.9264	
1	Random Forest	0.9786	0.9769	0.9799	0.9784	
2	Logistic Regression	0.9778	0.9682	0.9876	0.9778	
3	Linear SVM	0.9709	0.9552	0.9876	0.9711	
4	KNN	0.9725	0.9608	0.9845	0.9725	
5	Naive Bayes	0.8693	0.7924	0.9969	0.8830	
6	LDA	0.9404	0.9567	0.9212	0.9386	

Next steps:

[Generate code with results_df_rounded](#)

[New interactive sheet](#)

After training all of the models, I organized the results into a comparison table and a few simple visualizations so it would be easier to compare their performance side by side. This helped me move beyond looking at one model at a time and instead see the bigger picture across accuracy, precision, recall, and F1-score. Since intrusion detection is not just about overall accuracy, I especially wanted to

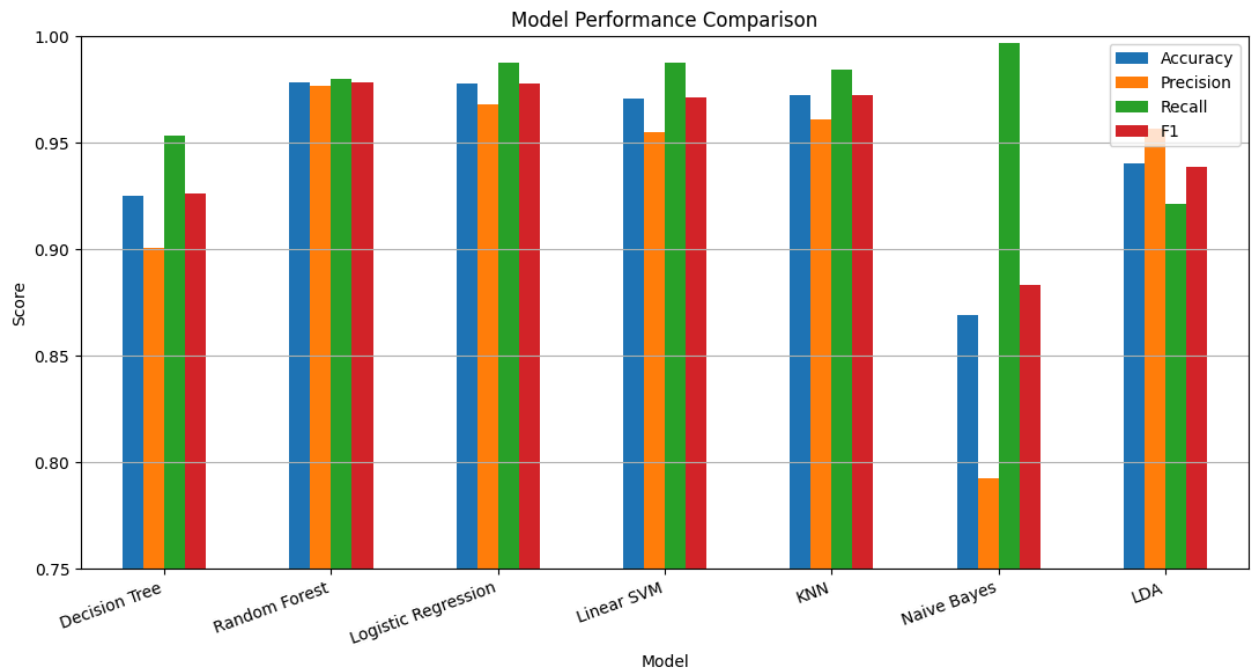
compare how well each model handled the trade-off between catching attacks and avoiding false alarms.

```
import matplotlib.pyplot as plt

plot_df = results_df_rounded.set_index("Model")

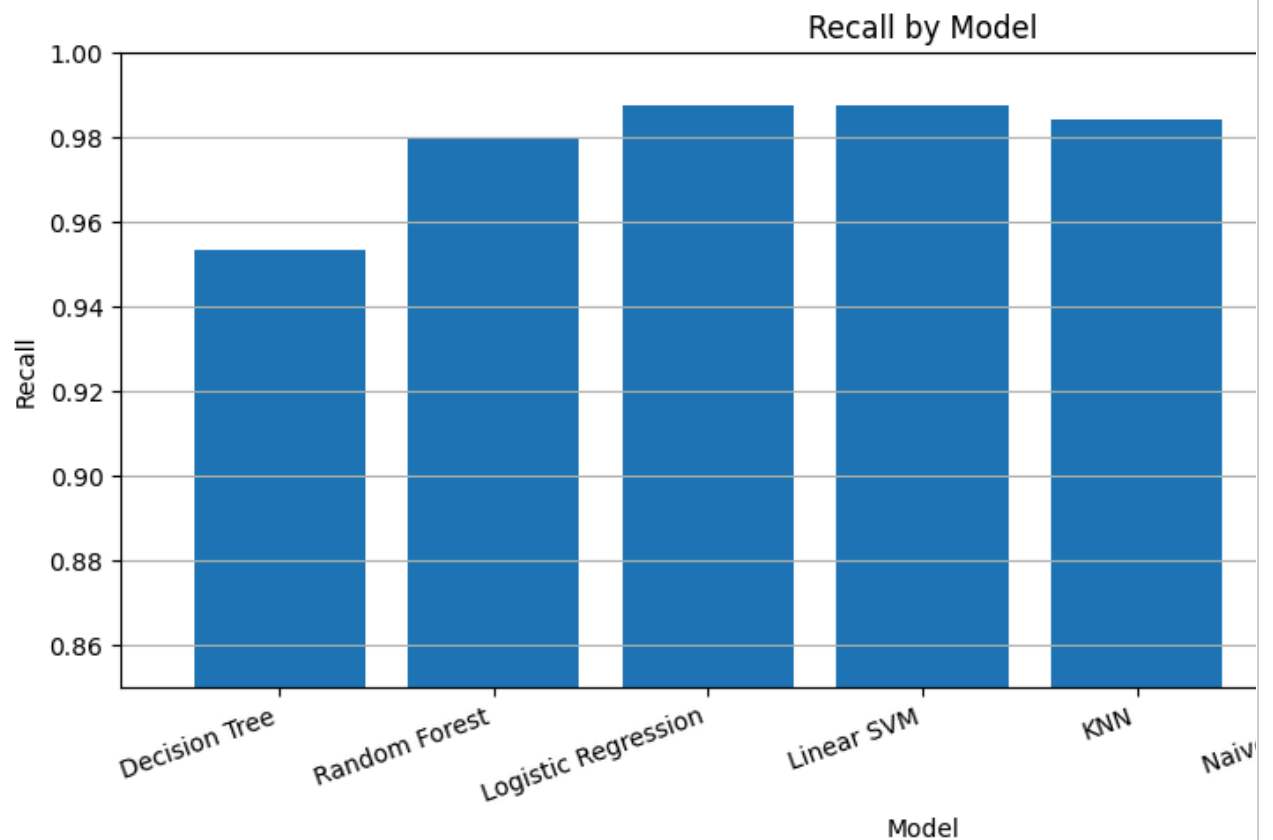
plot_df[["Accuracy", "Precision", "Recall", "F1"]].plot(kind="bar", figsize=(11, 6))

plt.title("Model Performance Comparison")
plt.ylabel("Score")
plt.xlabel("Model")
plt.xticks(rotation=20, ha="right")
plt.ylim(0.75, 1.00)
plt.grid(axis="y")
plt.tight_layout()
plt.show()
```



```
# Recall only table -- Important for Cybersecurity
plt.figure(figsize=(10, 5))
plt.bar(results_df_rounded["Model"], results_df_rounded["Recall"])

plt.title("Recall by Model")
plt.ylabel("Recall")
plt.xlabel("Model")
plt.xticks(rotation=20, ha="right")
plt.ylim(0.85, 1.00)
plt.grid(axis="y")
plt.tight_layout()
plt.show()
```



Exporting RF model to create API

```
import joblib

model_bundle = {
    "model": rf,
    "features": list(X_train.columns)
}

joblib.dump(model_bundle, "random_forest_ids.joblib")

print("Random Forest saved successfully!")
print("Number of features:", len(X_train.columns))
```

Random Forest saved successfully!
Number of features: 74

```
import sklearn
import joblib
import sys

print("Python:", sys.version)
print("scikit-learn:", sklearn.__version__)
print("joblib:", joblib.__version__)
```

Python: 3.13.15 (main, Aug 6 2026, 11:06:23) [GCC 11.4.0]
scikit-learn: 1.6.1
joblib: 1.5.3

```
print(X_test.shape)
print(y_test.shape)
```

```
(1308, 74)
(1308,)
```

```
sample = X_test.iloc[0]

print("Expected label:", y_test.iloc[0])
print(sample.to_dict())

sample.to_json("sample_flow.json")
```